

Approach Document

SHL Conversational Assessment Recommendation System

Problem Understanding

The system addresses the fundamental challenge of assessment selection in hiring: recruiters often lack precise vocabulary to search traditional catalogs, yet know what they need when they describe the role conversationally. Our solution bridges this gap through a RAG-based conversational agent that transforms vague intent into grounded, catalog-backed assessment recommendations.

Architecture Overview

The system implements a retrieval-augmented generation architecture with the following components:

- **FastAPI Backend:** Stateless REST endpoints (/health, /chat) designed for evaluator compatibility
- **SHL Catalog:** Normalized JSON dataset as the single source of truth, preventing hallucinations
- **Sentence Transformers:** Dense embeddings for semantic similarity (all-MiniLM-L6-v2 model)
- **FAISS Vector Store:** Fast nearest-neighbor retrieval with cosine similarity search
- **Deterministic Ranking Engine:** Heuristic reordering based on category match, role seniority, and business rules
- **Google Gemini:** Natural language generation with strict grounding constraints and deterministic fallback

Retrieval & Ranking Strategy

The system employs a two-phase approach: semantic retrieval followed by deterministic reranking. User messages are embedded and matched against catalog vectors using cosine similarity (top-20 retrieval). Retrieved candidates are then reranked by: (1) exact category match boosting, (2) role-level alignment (entry/mid/senior), and (3) test type compatibility rules (e.g., avoiding redundant cognitive tests). For battery composition, the system selects complementary assessments across categories (cognitive + personality + technical) while preventing overlap.

Conversation Intelligence

The agent implements four core behaviors:

- **Clarification:** Detects underspecified requests (e.g., 'need an assessment') and prompts for role, seniority, or skills before recommending
- **Recommendation:** Returns 1-10 catalog items with provenance (name, URL, test type) when sufficient context exists
- **Refinement:** Accepts mid-conversation edits ('add personality tests') and updates recommendations without restarting
- **Comparison:** Provides side-by-side assessment comparisons grounded in catalog metadata

Off-topic and legal requests are refused with safe, explanatory messages. The agent never discusses general hiring advice or topics outside SHL assessments.

Grounding & Hallucination Prevention

Preventing hallucinations is critical. All recommendations map to catalog records validated via Pydantic schemas. The Gemini prompt explicitly forbids invented assessments and includes only top-N retrieval snippets as context. If grounding confidence is low or Gemini fails, the system returns a deterministic fallback: the ranked catalog output with a fixed-format reply. This ensures every response is traceable and evaluator-reproducible.

API Design

The API follows strict schema compliance for automated evaluation. POST /chat accepts a stateless 'messages' array (role/content) and returns 'reply' (natural language), 'recommendations' (validated catalog items), and 'end_of_conversation' flag. Recommendations are empty during clarification/refusal, contain 1-10 items when committed. Each recommendation includes 'name', 'url', and 'test_type'. The design is intentionally batchable for evaluator harnesses.

Evaluation Approach

I developed and tested against multiple scenarios:

- **Vague queries:** Confirmed clarification prompts before recommendations
- **Technical hiring:** Verified role-specific prioritization (e.g., Java developer → Java 8 test)
- **Graduate hiring:** Validated battery composition across cognitive, personality, and aptitude categories
- **Comparison requests:** Ensured side-by-side evidence with catalog provenance
- **Legal/off-topic:** Confirmed safe refusal without catalog leakage

Grounding checks confirm every recommendation maps to a catalog ID. Deterministic outputs are reproducible with fixed inputs. Latency targets are met for interactive use (retrieval + ranking < 1s; Gemini response considered separately).

Technical Stack & Deployment

Stack: Python 3.11, FastAPI, Sentence Transformers (all-MiniLM-L6-v2), FAISS, Google Gemini 1.5 Flash, Pydantic.
Deployment: Render web service with environment-based API key configuration and /health endpoint for cold-start readiness.

Challenges & Trade-offs

Semantic vs. Deterministic: Semantic search improves recall but may surface borderline items. Deterministic heuristics guard precision and evaluator expectations. The hybrid approach balances both.

LLM Reliability: Gemini provides natural responses but introduces variability. The deterministic fallback ensures stability for automated evaluation, trading some conversational fluency for trust and reproducibility.

Avoiding Over-generation: Generator prompts are tightly constrained with explicit 'do not invent' instructions and limited to retrieval context only. This reduces naturalness but increases grounding confidence.

What Didn't Work

Pure LLM Router: Initially attempted intent classification via LLM alone, but found it unreliable for edge cases. Switched to deterministic pattern matching with LLM augmentation for ambiguous cases.

Larger Embedding Models: Tested larger sentence transformers (mpnet-base) but found minimal accuracy gain with 3x latency increase. Chose all-MiniLM-L6-v2 for speed-accuracy balance.

Use of AI Tools

I used AI-assisted coding (GitHub Copilot, Claude) for boilerplate generation, API schema design, and initial prompt templates. Core logic (ranking heuristics, conversation flow, grounding verification) was written manually with clear understanding of trade-offs. All code was reviewed and tested iteratively.

Future Improvements

- Persist FAISS index and separate embedding worker for scalability
- Add per-recommendation confidence scores and field-level catalog citations
- Implement A/B evaluation harness comparing Gemini vs. fallback on labeled data
- Incorporate role-specific tuning for verticals (sales, engineering, leadership)

Document generated: May 10, 2026